Design and Analysis of Algorithms
(Course Code: 25TU03MJM1)

PROJECT BASED LEARNING REPORT



Title of the Project
(**Smart Emergency Route and Resource Optimization Using Advanced Algorithms**)

Semester: 3
Academic Year: 2026-27

Submitted by:
D. Keerthana (25WU0102125)
K. Rohan Chowdary (25WU0102122)
K. Harshitha (25WU0102134)
S. Pranathi (25WU0104024)
Section: AI&ML (Panthers)

Under the Guidance of
Prof. Geetha Tripathi

School of Technology

CERTIFICATE

This is to certify that the Project titled **"Smart Emergency Route and Resource Optimization Using Advanced Algorithms"** is a bonafide work carried out by **D. Keerthana (Register No: 25WU0102125)**, K. Rohan Chowdary (25WU0102122), K. Harshitha (25WU0102134) and S. Pranathi (25WU0104024), B.Tech – Computer Science and Engineering, during the Academic Year **2026-27**, under my supervision and guidance.

Guide Signature
Prof. Geetha Tripathi

DECLARATION

We hereby declare that the Project titled "Smart Emergency Route and Resource Optimization Using Advanced Algorithms" submitted by us is an original work carried out under the guidance of the faculty mentioned above. This work has not been submitted in part or full for the award of any degree, diploma, or other academic qualification in this or any other institution. All sources of information used have been properly acknowledged and cited as per university referencing standards.

Place: Woxsen University Date:

Signature of the Candidates
D. Keerthana (25WU0102125)
K. Rohan Chowdary (25WU0102122)
K. Harshitha (25WU0102134)
S. Pranathi (25WU0104024)

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide for the continuous guidance, encouragement and constructive feedback given throughout this project. We are thankful to the Head of the Department of Computer Science and Engineering and the faculty of the School of Technology, Woxsen University, for providing the academic environment and resources needed to complete this work. We also thank our classmates and families for their support and encouragement.

1. ABSTRACT

In an emergency, the time taken by an ambulance, fire engine or police unit to reach the scene decides how much harm is done. In real cities this time is affected by congestion, blocked roads and the choice of which unit is sent to which incident. This project presents **RescueRoute AI**, an interactive web application that models a city as a weighted graph and uses classical algorithms from Design and Analysis of Algorithms to find the fastest emergency route and to allocate response units to incidents.

The system compares four shortest-path algorithms (Dijkstra, A*, Bellman-Ford and Breadth-First Search) on a simulated road network with congestion and road closures, and compares three allocation strategies (random, greedy nearest-unit and Hungarian optimal assignment). Experiments on simulated networks show that Dijkstra, A* and Bellman-Ford return identical optimal routes, A* expands 52 % fewer nodes than Dijkstra, and Bellman-Ford is about 11 times slower. BFS produces routes that are 20 % slower because it ignores travel time. Closing 30 % of the roads raises the fastest travel time by 25 %. Optimal assignment cuts the total response time by 11–15 % compared with greedy dispatch and by 41–49 % compared with random dispatch. The results come from simulation and need validation on real city data.

Keywords: *emergency routing, shortest path, Dijkstra, A*, Hungarian algorithm, resource allocation, road closures*

TABLE OF CONTENTS

2. INTRODUCTION....1
2.1 Background....1
2.2 Motivation and Relevance to Real-World Problems....1
2.3 Scope and Organisation of the Report....1
3. PROBLEM STATEMENT AND OBJECTIVES....1
3.1 Problem Definition....1
3.2 Objectives....1
3.3 Constraints and Assumptions....2
4. LITERATURE REVIEW....2
4.1 Review of Related Work....2
4.2 Comparison of Related Work....3
5. METHODOLOGY AND SYSTEM DESIGN....3
5.1 Overall System Architecture....3
5.2 Flowchart and Data Flow Diagram....4
5.3 Algorithm Explanation....4
5.4 Tools and Technologies....5
6. IMPLEMENTATION....5
6.1 Module-wise Implementation....5
6.2 Dataset and Input Description....6
6.3 Pseudocode....6
6.4 Screenshots and Output....7
7. RESULTS AND DISCUSSION....7
7.1 Experimental Setup....7
7.2 Output Tables....7
7.3 Performance Graphs....8
7.4 Result Interpretation....9
7.5 Comparison with Objectives....9
8. CONCLUSION AND FUTURE SCOPE....9
8.1 Summary....9
8.2 Learning Outcomes....10
8.3 Future Enhancements....10
REFERENCES....10

LIST OF FIGURES

Figure 5.1: System architecture of RescueRoute AI....3
Figure 5.2: Flowchart of the system....4
Figure 5.3: Data flow diagram (level 0)....4
Figure 6.1: Simulated city network with closed roads and routes....7
Figure 6.2: Dashboard of the live application....7
Figure 7.1: Comparison of the routing algorithms....8
Figure 7.2: Runtime growth and effect of road closures....8
Figure 7.3: Total response time by allocation strategy....9

LIST OF TABLES

Table 3.1: Constraints and assumptions....2
Table 4.1: Comparison of related work and the proposed system....3
Table 5.1: Algorithms used and their complexity....5
Table 5.2: Tools and technologies....5
Table 6.1: Modules of the system....5
Table 6.2: Input and output data fields....6
Table 7.1: Routing algorithm comparison (400 intersections)....7
Table 7.2: Effect of road closures (Dijkstra)....8
Table 7.3: Resource allocation results....8
Table 7.4: Achievement of project objectives....9

2. INTRODUCTION

2.1 Background

Emergency response is a race against time. For a cardiac arrest or a building fire, every extra minute on the road lowers the chance of survival and increases damage. The delay does not come only from distance. Congestion slows some roads, accidents and floods close others, and a dispatcher must also decide which of several available units should go to which incident. These are the two decisions that this project automates: **which route to take** and **which unit to send**.

Both decisions are graph problems. A city map is a weighted graph in which intersections are vertices, roads are edges and the travel time of a road is its weight. Finding the fastest route is a shortest-path problem, and matching units to incidents is an assignment problem. Algorithms for both are standard topics in Design and Analysis of Algorithms (DAA), which makes this domain a good test bed for comparing their correctness, running time and practical behaviour. The project is delivered as a web application, **RescueRoute AI** [11].

2.2 Motivation and Relevance to Real-World Problems

Navigation tools optimise for ordinary drivers, not for emergency fleets that must re-plan when a road closes and must share a limited number of units among simultaneous incidents. A dispatcher choosing the nearest unit by intuition can leave a second incident badly served. A tool that shows how different algorithms behave, and how much each decision costs in minutes, helps students and planners understand why algorithm choice matters in safety-critical systems and in smart-city control rooms.

2.3 Scope and Organisation of the Report

The scope covers a simulated city network with congestion and road closures, four routing algorithms, three allocation strategies, and a performance comparison. Real-time traffic feeds, live GPS tracking and multi-objective vehicle routing are outside the scope and are listed as future work. The report is organised as follows: Chapter 3 states the problem and objectives, Chapter 4 reviews related work, Chapter 5 presents the design and algorithms, Chapter 6 describes the implementation, Chapter 7 reports the results, and Chapter 8 concludes.

3. PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem Definition

Let the road network be a weighted graph $G = (V, E, w)$, where V is the set of intersections, E the set of roads, and $w(e)$ the current travel time of road e in minutes. Let $C \subseteq E$ be the set of closed roads, U the set of available response units, and I the set of active incidents with $|U| \geq |I|$.

The problem is to find, for every incident, the minimum-time route over the open roads $E \setminus C$, and to assign units to incidents so that the total (and the severity-weighted) response time is as small as possible, while comparing the algorithms by correctness, speed and growth with network size.

3.2 Objectives

- **Model the city** as a weighted graph with congestion and closable roads.
- **Implement and compare** Dijkstra, A*, Bellman-Ford and BFS for emergency routing.

- **Handle road closures** by re-planning routes and measuring the added travel time.
- **Allocate resources** using greedy and Hungarian assignment and compare them with random dispatch.
- **Analyse performance** in terms of travel time, nodes explored, runtime and complexity, and present the results in an interactive dashboard.

3.3 Constraints and Assumptions

Table 3.1 lists the constraints under which the reported results are valid.

Table 3.1: Constraints and assumptions

Category	Description
Network	Simulated city: grid-like road graph with random diagonal links; 100 to 2,500 intersections.
Travel time	Free-flow time equals road length (1 unit = 1 minute); congestion multiplies it by 1.0–2.5.
Closures	Closed roads are removed from the graph; the network is kept connected.
Resources	Equal number of units and incidents; one unit serves one incident; units are idle at start.
Severity	Incidents have severity 1 (minor) to 3 (critical) used only for weighted dispatch.
Evaluation	Simulation only; no field data, live traffic or real map has been used.

4. LITERATURE REVIEW

4.1 Review of Related Work

Dijkstra [1] gave the greedy shortest-path algorithm for non-negative weights; with a binary heap it runs in O((V+E) log V) and Fibonacci heaps improve it further [9]. **Hart, Nilsson and Raphael [2]** introduced A*, which adds an admissible heuristic (here straight-line distance) so that fewer vertices are explored while the route stays optimal. **Bellman [3]** described the dynamic-programming relaxation method that handles negative weights in O(V·E), and **Floyd [8]** gave the $O(V^3)$ all-pairs method.

For assignment, **Kuhn [4]** and **Munkres [10]** developed the Hungarian method, which solves the minimum-cost matching problem in $O(n^3)$. In emergency services, **Brotcorne, Laporte and Semet [5]** surveyed ambulance location and relocation models and showed that where units are and how they are dispatched strongly affects response time. **Bast et al. [6]** review practical route planning in road networks and show that goal-directed search such as A* is a core technique. Standard textbook treatments are given in [7].

4.2 Comparison of Related Work

Table 4.1: Comparison of related work and the proposed system

Work	Problem	Strength	Limitation
Dijkstra [1]	Shortest path	Optimal, simple	Explores in all directions
A* [2]	Shortest path	Fewer vertices explored	Needs admissible heuristic
Bellman-Ford [3]	Shortest path	Negative weights allowed	O(V·E), slow
Hungarian [4],[10]	Assignment	Optimal matching	$O(n^3)$ for n units
Brotcorne et al. [5]	Ambulance location	Domain models	Not an algorithm comparison
RescueRoute AI	Routing + dispatch	Side-by-side comparison with closures	Simulation-based, single type of unit

5. METHODOLOGY AND SYSTEM DESIGN

5.1 Overall System Architecture

The system is a six-stage pipeline (Figure 5.1). The input layer supplies the road network, incidents and unit locations. The preprocessing stage builds the weighted graph and applies congestion and closures. The routing engine runs the selected shortest-path algorithm. Resource allocation matches units to incidents using the route times. Performance analysis records travel time, nodes explored and runtime, and the dashboard shows the result. New incidents or closures feed back into the input layer.

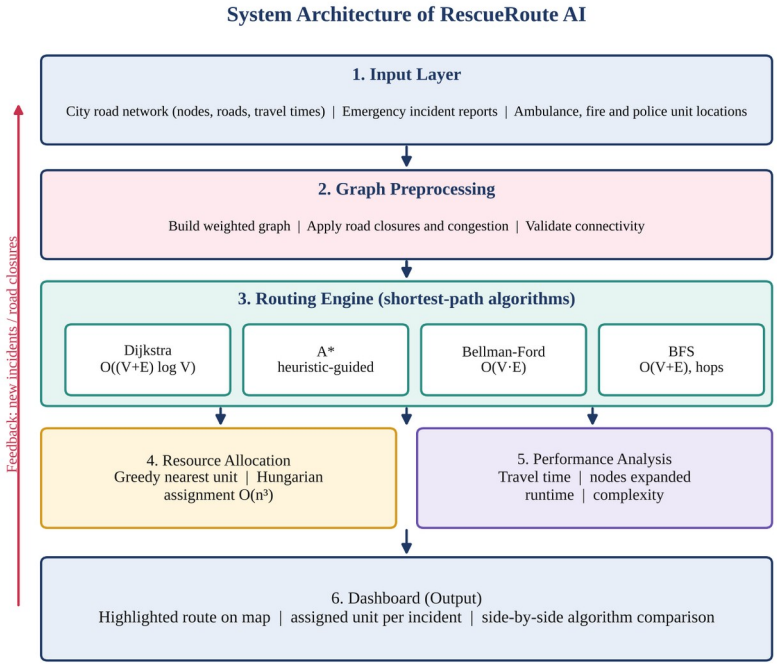


Figure 5.1: System architecture of RescueRoute AI

5.2 Flowchart and Data Flow Diagram

Figure 5.2 shows the control flow of one dispatch cycle and Figure 5.3 shows the level-0 data flow between the dispatcher, the four processes and the two data stores.

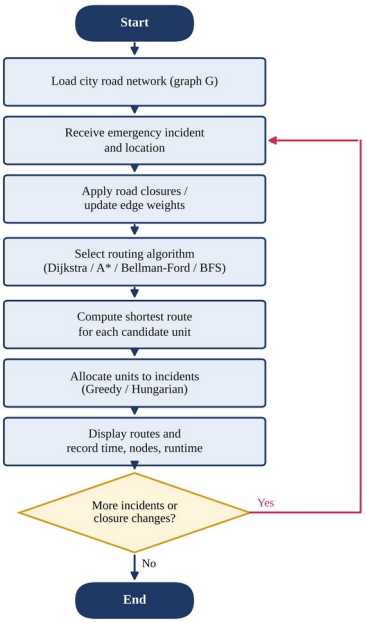


Figure 5.2: Flowchart of the system

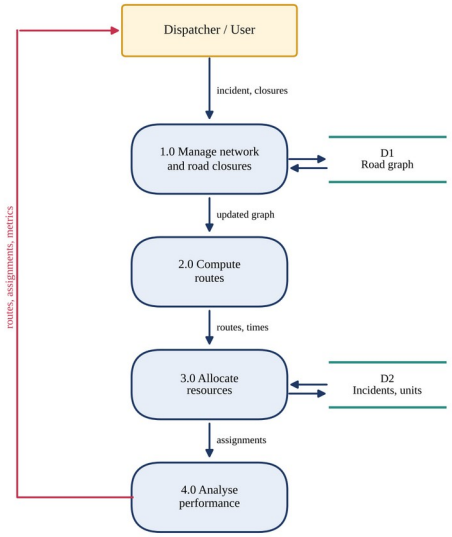


Figure 5.3: Data flow diagram (level 0)

5.3 Algorithm Explanation

- **Dijkstra.** Repeatedly settles the unsettled vertex with the smallest known distance using a min-heap and relaxes its outgoing roads. It stops as soon as the incident vertex is settled.
- **A*.** Same as Dijkstra, but vertices are ordered by f(v) = g(v) + h(v), where h(v) is the straight-line distance to the incident. Because every road takes at least as long as its length, h never overestimates, so the route is still optimal.
- **Bellman-Ford.** Relaxes every road up to V−1 times, stopping early when nothing changes. It does not need a priority queue, but it does far more work.
- **BFS.** Explores the graph level by level and finds the route with the fewest roads, ignoring travel time. It is used as an unweighted baseline.
- **Greedy and Hungarian dispatch.** Greedy gives each incident the nearest free unit; priority greedy serves critical incidents first. The Hungarian method builds the cost matrix of route times and finds the assignment with the smallest total cost; multiplying each column by severity gives a severity-weighted version.

Table 5.1: Algorithms used and their complexity

Algorithm	Time	Space	Role in the system
Dijkstra	O((V+E) log V)	O(V)	Reference fastest-route algorithm
A*	O((V+E) log V) worst case	O(V)	Goal-directed fastest route
Bellman-Ford	O(V·E)	O(V)	Slower alternative; negative weights
BFS	O(V+E)	O(V)	Fewest-roads baseline
Floyd–Warshall	$O(V^3)$	$O(V^2)$	All-pairs timing comparison
Hungarian	$O(n^3)$	$O(n^2)$	Optimal unit-to-incident assignment

5.4 Tools and Technologies

Table 5.2: Tools and technologies

Item	Used for
Replit (hosting)	Hosts the live RescueRoute AI web application [11]
Web front end	Network display, closure controls, route highlighting and algorithm comparison
Python 3, NumPy, SciPy, Matplotlib	Reference implementation, experiments and charts in this report

6. IMPLEMENTATION

6.1 Module-wise Implementation

The application is divided into the modules in Table 6.1. Each module has one job, so an algorithm can be replaced without changing the interface.

Table 6.1: Modules of the system

Module	Function
Network builder	Creates the intersections and roads, assigns travel times and congestion.
Closure manager	Lets the user close or reopen roads and rebuilds the graph.
Routing engine	Runs Dijkstra, A*, Bellman-Ford or BFS and returns the route and statistics.
Resource allocator	Assigns units to incidents using greedy or Hungarian matching.
Performance analyser	Reports travel time, vertices explored, runtime and complexity.
Dashboard	Shows the map, highlighted routes and side-by-side comparison.

6.2 Dataset and Input Description

No external dataset is required. The city is generated as a jittered grid of intersections with horizontal and vertical roads and a random 25 % of diagonal roads. Each road receives a congestion factor between 1.0 and 2.5. Table 6.2 gives the data fields.

Table 6.2: Input and output data fields

Entity	Fields	Description
Intersection	id, x, y	Vertex of the graph with map coordinates
Road	from, to, time, open	Edge with travel time in minutes and closed/open state
Incident	location, severity (1–3)	Emergency that needs a unit
Unit	type, location	Ambulance, fire or police unit available for dispatch
Output	route, time, vertices explored, runtime	Result of the chosen algorithm

6.3 Pseudocode

```
DIJKSTRA(G, s, t):                    // closed roads already removed
  dist[v] = INF for all v; dist[s] = 0; PQ = {(0, s)}
  while PQ not empty:
    (d, u) = EXTRACT-MIN(PQ)
    if u == t: return dist[t], PATH(prev, t)
    for each road (u, v, w) in G:
      if dist[u] + w < dist[v]:
        dist[v] = dist[u] + w; prev[v] = u; INSERT(PQ, (dist[v], v))
  return NO-ROUTE
```

```
DISPATCH(G, units, incidents, mode):
  for each unit i and incident j: cost[i][j] = DIJKSTRA(G, i, j)
  if mode == GREEDY:    for each j: assign nearest free unit
  if mode == HUNGARIAN: assignment = MIN-COST-MATCHING(cost)
  return assignment, total response time
```

6.4 Screenshots and Output

Figure 6.1 shows a simulated city of 144 intersections with 20 % of roads closed. The fastest route (Dijkstra and A*, 28.2 min) detours around closures, whereas BFS picks the route with the fewest roads and takes 30.7 min. Figure 6.2 is reserved for a screenshot of the live application.

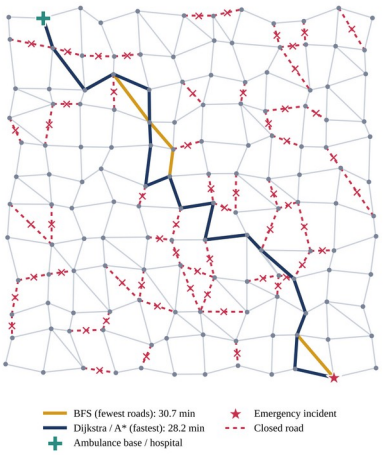


Figure 6.1: Simulated city network with closed roads and routes

[Insert screenshot of the RescueRoute AI dashboard here]

Figure 6.2: Dashboard of the live application

7. RESULTS AND DISCUSSION

7.1 Experimental Setup

The algorithms were implemented in Python and run on simulated road networks. The main comparison uses five random 20 × 20 networks (400 intersections, about 850 roads) and 20 random incident–source pairs per network, giving 100 routing queries. Scaling was tested from 100 to 2,500 intersections. Allocation was tested on 12 × 12 networks over 60 random scenarios per size, with 10 % of roads closed. Runtimes are averages on one machine and show relative behaviour, not absolute speed.

7.2 Output Tables

Table 7.1: Routing algorithm comparison (400 intersections)

Algorithm	Avg. travel time (min)	Vertices explored / roads relaxed	Runtime (ms)	Optimal route?
Dijkstra	19.28	211.6	0.249	Yes
A*	19.28	101.1	0.183	Yes
Bellman-Ford	19.28	32,122	2.84	Yes
BFS	23.15	210.4	0.116	No (+20.1 %)

Table 7.2: Effect of road closures (Dijkstra)

Roads closed	Avg. fastest route (min)	Increase over open network	Vertices explored
0 %	19.28	–	211.6
10 %	20.01	+3.8 %	208.9
20 %	21.61	+12.1 %	209.9
30 %	24.06	+24.8 %	208.8

Table 7.3: Resource allocation results (average over 60 scenarios, minutes)

Strategy	Total (6 units)	Max (6 units)	Total (10 units)	Max (10 units)	Severity-weighted (10)
Random	77.3	20.0	120.8	21.4	252.9
Greedy nearest	51.7	16.0	73.3	17.2	154.4
Priority greedy	51.8	16.1	75.6	17.7	137.2
Hungarian (optimal)	**45.9**	**13.6**	**62.2**	**13.7**	131.5
Hungarian (severity-weighted)	47.1	14.3	64.8	14.8	**123.7**

7.3 Performance Graphs

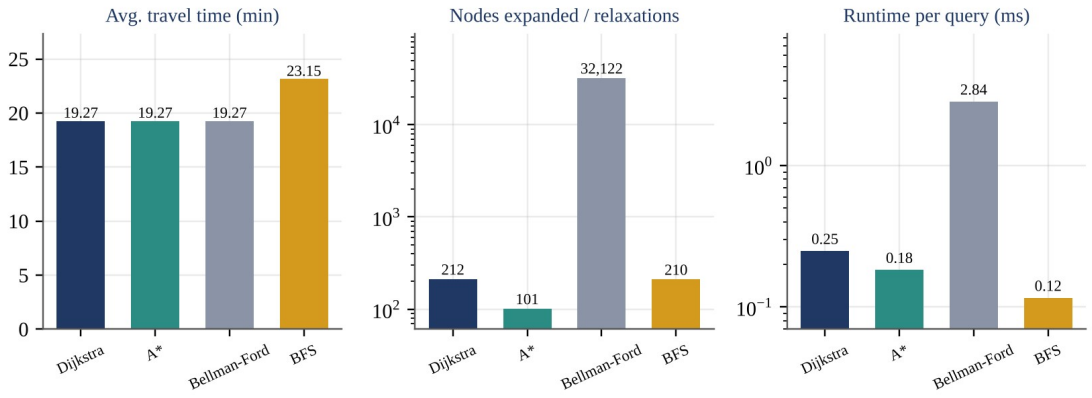


Figure 7.1: Comparison of the routing algorithms (log scale in the middle and right charts)

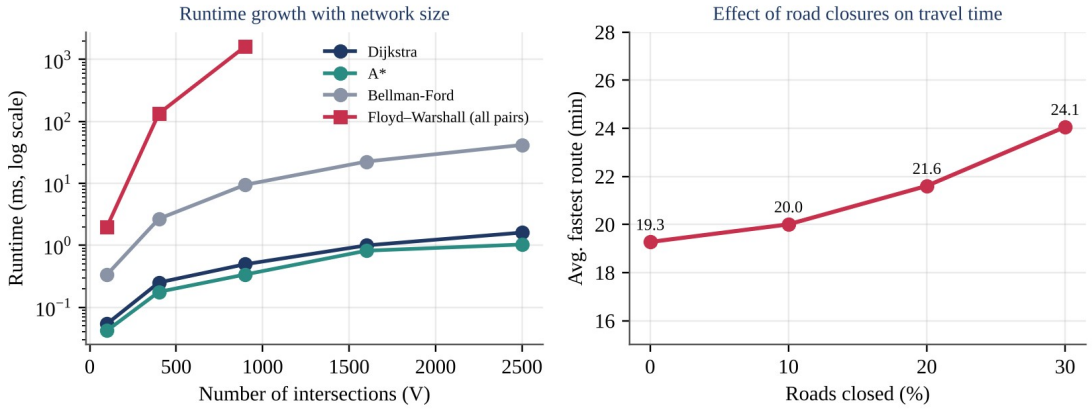


Figure 7.2: Runtime growth with network size (left) and effect of road closures (right)

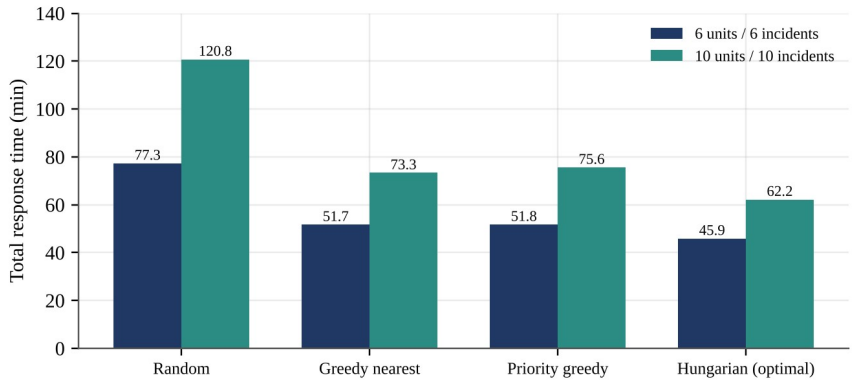


Figure 7.3: Total response time by allocation strategy

7.4 Result Interpretation

- **Correctness.** Dijkstra, A* and Bellman-Ford returned the same optimal travel time in all 100 queries, which confirms that the A* heuristic was admissible.
- **Efficiency.** A* explored 52 % fewer vertices than Dijkstra and ran 27 % faster. Bellman-Ford relaxed about 150 times more roads and was 11 times slower. From 400 to 2,500 intersections Dijkstra grew about 6.4 times (almost linear), Bellman-Ford 15.5 times, and Floyd–Warshall grew about 815 times from 100 to 900 intersections, in line with $O(V^3)$.
- **Why BFS fails.** BFS was fastest to run but its routes were 20 % slower, because the route with the fewest roads is not the fastest when congestion differs between roads.
- **Closures.** Each extra 10 % of closed roads added more delay than the last (3.8 %, 12.1 %, 24.8 %), so re-planning matters most when many roads are lost.
- **Dispatch.** Hungarian assignment cut the total response time by 11.3 % (6 units) and 15.2 % (10 units) against greedy, and the longest single response by 15–20 %. The severity-weighted version cut the weighted time by 20 % against greedy for 10 units while allowing a slightly higher plain total.

7.5 Comparison with Objectives

Table 7.4: Achievement of project objectives

Objective	Outcome
Model the city as a graph	Achieved: weighted graph with congestion and closures (Figure 6.1).
Compare routing algorithms	Achieved: four algorithms compared on time, effort and optimality (Table 7.1).
Handle road closures	Achieved: re-planning and delay measured (Table 7.2).
Allocate resources	Achieved: Hungarian beats greedy and random (Table 7.3).
Analyse performance and build dashboard	Achieved: complexity, scaling and a live web application [11].

8. CONCLUSION AND FUTURE SCOPE

8.1 Summary

The project showed how classical algorithms support emergency response. Dijkstra and A* give optimal routes, A* with far less work; Bellman-Ford is correct but costly; BFS is unsuitable when travel times differ. Road closures raise travel time sharply, and optimal

assignment reduces the total response time compared with greedy and random dispatch. The findings come from simulated networks and should be checked against real city data.

8.2 Learning Outcomes

- Applied graph algorithms and analysed their time and space complexity on a realistic problem.
- Compared theory with measured behaviour, including heuristics, scaling and the cost of optimal matching.
- Built an interactive tool and presented results with tables and graphs.

8.3 Future Enhancements

- Use real road maps and live traffic data, with time-dependent travel times.
- Handle more incidents than units, several unit types and hospital capacity.
- Add dynamic re-routing while a unit is on the way, and bidirectional or contraction-hierarchy search for very large cities.

REFERENCES

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[2] P. E. Hart, N. J. Nilsson and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Trans. Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.

[3] R. Bellman, "On a routing problem," Quarterly of Applied Mathematics, vol. 16, no. 1, pp. 87–90, 1958.

[4] H. W. Kuhn, "The Hungarian method for the assignment problem," Naval Research Logistics Quarterly, vol. 2, pp. 83–97, 1955.

[5] L. Brotcorne, G. Laporte and F. Semet, "Ambulance location and relocation models," European Journal of Operational Research, vol. 147, no. 3, pp. 451–463, 2003.

[6] H. Bast et al., "Route planning in transportation networks," in Algorithm Engineering, Lecture Notes in Computer Science, vol. 9220, Springer, 2016, pp. 19–80.

[7] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[8] R. W. Floyd, "Algorithm 97: Shortest path," Communications of the ACM, vol. 5, no. 6, p. 345, 1962.

[9] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," Journal of the ACM, vol. 34, no. 3, pp. 596–615, 1987.

[10] J. Munkres, "Algorithms for the assignment and transportation problems," Journal of the SIAM, vol. 5, no. 1, pp. 32–38, 1957.

[11] RescueRoute AI web application, https://rescue-route-ai--daramanishwar.replit.app (accessed Oct. 2026).